Proof-Guided SAT Encoding Selection for Multiplication in Bounded Model Checking

Michael Tautschnig[1], TBD[1]

[1]*Amazon Web Services*

Abstract

We present a methodology for designing SAT encodings of integer multiplication guided by proof trace analysis. By examining DRAT proofs, we identify structural bottlenecks—the top bottleneck variable in standard Comba encoding appears in 31% of all proof steps—and use these findings to redesign the encoding. This methodology produced COMBA-CS, a carry-save Comba encoding that separates column reduction from carry propagation, achieving 2.6–71× speedup across four SAT solvers. We further show that carry propagation—not circuit size—is the dominant hardness source: at 13 bits, integer multiplication needs 23× more conflicts than carry-free GF(2) multiplication despite only 1.6× more clauses. Applying the same methodology to alternative encodings (Booth radix-4, 4-bit blocks, sorting networks), we find that no single encoding dominates: Booth is 17× faster on constant multiplication, while COMBA-CS is 2× faster on commutativity. The optimal encoding depends on problem structure, motivating adaptive selection.

Keywords

SAT encoding, bounded model checking, multiplication, DRAT proof analysis, bounded variable elimination

1. Introduction

Consider verifying that 16-bit multiplication commutes: does $a \times b = b \times a$ hold for all 16-bit inputs a and b? A bounded model checker encodes this as a SAT problem with two multiplier circuits and an equality check. With the standard shift-and-add encoding, CaDiCaL [1] times out after 120 s. With our carry-save Comba encoding, it solves in 5.3 s—a 71× speedup from changing only the encoding.

How did we arrive at this encoding? Not by theoretical analysis of circuit complexity, but by a systematic methodology: generate a DRAT proof of the existing encoding, identify which variables participate most heavily in the proof, trace those variables back to the encoding structure, and redesign to reduce the identified bottlenecks. This proof-guided approach is transferable to other encoding problems and is the primary contribution of this paper.

SAT-based bounded model checking (BMC) [2] encodes program semantics as Boolean formulas, and for programs involving integer multiplication—common in cryptography, DSP, and overflow checking—the encoding choice dramatically affects solver performance. Yet the interaction between encoding choices and modern SAT solver techniques (inprocessing [3], BVE [4]) is poorly understood. Prior work has focused on BDD-based complexity [5] and propagation completeness [6, 7], but not on systematic evaluation of SAT encodings across multiple solvers with solver-level proof analysis.

Our contributions are:

1. **Proof-guided encoding methodology.** We demonstrate a systematic approach: DRAT proof analysis → bottleneck identification → encoding redesign → validation. This produced COMBA-CS and can be applied to other encoding problems (Section 4).
2. **Carry propagation as the hardness source.** We compare integer and GF(2) (carry-less) multiplication and show that carry propagation—not circuit size—causes exponential blowup (Section 3).
3. **Six-encoding comparison with solver statistics.** We evaluate shift-add, Dadda, Comba, COMBA-CS, Booth radix-4, 4-bit blocks, and sorting networks across four SAT solvers, reporting conflicts, propagations, BVE eliminations, fixed variables, and proof sizes. No single encoding dominates (Sections 5, 6, 7).

0000-0002-7947-983X (M. Tautschnig)

4. **Negative results.** We catalogue 20+ approaches that failed, including subquadratic algorithms, XOR Gaussian elimination, and abstraction refinement (Section 8).

2. Background

An N-bit unsigned multiplication $a \times b$ produces N partial products $pp[i] = a[i] \wedge b$, each shifted by i positions. These must be accumulated into the final $2N$-bit product. The standard *shift-add* encoding adds partial products sequentially using a ripple-carry adder chain, creating a carry chain of length $O(N^2)$. *Dadda trees* [8] reduce the partial product matrix to two rows using carry-save full adders, followed by a single carry-propagate addition. *Comba* [9] reduces each column independently using popcount (parallel bit counting via the pop0 algorithm from [10]), propagating carries to adjacent columns during the same pass. Section 5 introduces our carry-save variant that decouples these two steps.

Modern CDCL solvers employ techniques that interact with encoding design in ways we exploit throughout this paper. Bounded Variable Elimination (BVE) [4] eliminates variables by resolving all clauses containing them; it is effective when the resolvent count is small. CaDiCaL [1] performs BVE during search (*inprocessing*), while MiniSat/SatELite [4] performs it only during preprocessing. CaDiCaL also extracts XOR and AND gates from CNF and identifies congruent (functionally equivalent) gate pairs—for BW=9 commutativity, it finds 131 XOR gates, 138 AND gates, and 130 congruent pairs between two multiplier circuits.

Brain et al. [6] showed that composition of propagation-complete (PC) primitives preserves PC for adders but *not* for multipliers. Brain [7] conjectured that a PC multiplier encoding is exponential in size. Rather than seeking propagation completeness, we optimize encoding structure for interaction with BVE and inprocessing.

3. Carry Propagation Hardness

We compare integer multiplication commutativity ($a \times b = b \times a$) with GF(2) (carry-less) multiplication commutativity. Both have identical partial product grids; the only difference is carry propagation (integer uses addition with carries, GF(2) uses XOR). This comparison isolates carry propagation as a variable, though we note that the change in addition operation also affects clause structure and propagation properties.

Table 1
Integer vs. GF(2) multiplication commutativity (CaDiCaL).

	GF(2)			Integer		
BW	Clauses	Conflicts	Time	Clauses	Conflicts	Time
9	1,915	974	0.01s	2,421	10,831	0.22s
13	3,059	10,707	0.08s	4,993	250,630	8.66s

At BW=13, integer multiplication needs 23× more conflicts than GF(2) despite only 1.6× more clauses. Empirically, GF(2) conflict count grows roughly as n^3 (measured across BW=5–17); integer multiplication grows exponentially. DRAT proof analysis confirms: GF(2) proofs are 3.6–18× smaller.

This is consistent with known exponential resolution lower bounds for related problems [11] and with Lv and Kalla's finding [12] that GF(2^k) multipliers (carry-free) are verifiable algebraically but intractable for SAT beyond 16 bits. We emphasize that this is empirical evidence; a formal proof of exponential resolution complexity for multiplication remains open. The practical implication is clear: encodings that minimize carry propagation depth should reduce SAT solver effort. Corroborating evidence comes from the encoding comparison itself (Table 5): the encoding hierarchy shift-add <

Dadda < Comba < COMBA-CS exactly mirrors decreasing carry propagation depth, with all four using the same partial products and integer addition. This is exactly what COMBA-CS does.

4. Proof-Driven Encoding Design

Our encoding improvements were not discovered by theoretical analysis but by a systematic methodology that examines solver behavior:

1. **Generate DRAT proof** of the existing encoding on a representative benchmark.
2. **Identify bottleneck variables**: measure which variables appear most frequently in proof steps.
3. **Trace to encoding structure**: map bottleneck variables back to the multiplier circuit to understand which encoding decisions cause the bottleneck.
4. **Redesign encoding** to reduce the identified bottleneck.
5. **Validate** across multiple solvers and benchmarks.

Applying this to standard Comba on commutativity BW=11, we found that the top bottleneck variable appears in 31% of all DRAT proof steps. This variable corresponds to a carry bit at the boundary between two columns—the point where standard Comba propagates carries from one column's popcount to the next. This inter-column dependency forces the solver to reason about carry propagation across the entire width, creating a sequential bottleneck in an otherwise parallel structure.

The fix was COMBA-CS: separate column reduction (first pass) from carry propagation (second pass). The result: 55% smaller proofs, 74% more unit propagation during search, and 2.6–71× speedup (Table 2).

This methodology is transferable. We applied it to identify why Booth radix-4 excels on constant multiplication (Section 6): proof analysis shows that Booth's zero-valued digits for constants with few set bits eliminate entire partial products, reducing conflicts by 10×. Similarly, proof analysis of the sorting network encoding reveals that its 2,915 fixed variables (from auxiliary sorted-property constraints) are overwhelmed by the 13K extra variables—a net negative despite excellent propagation properties.

The methodology could be automated: an AI system that reads DRAT proofs, identifies structural bottlenecks, and proposes encoding modifications would close the loop that we perform manually. No existing work automates this full cycle—prior work either analyzes encoding *code* (not solver output) [3] or modifies solver *heuristics* (not encodings) [1].

5. Carry-Save Comba Encoding

Standard Comba processes columns left-to-right, propagating carry bits from each column's popcount to the next column during the same pass. This creates inter-column dependencies that manifest as proof bottlenecks: the top bottleneck variable appears in 31% of all DRAT proof steps.

COMBA-CS separates the two concerns:

1. **First pass:** Reduce each column independently via popcount. Carry bits are collected but *not* propagated to adjacent columns.
2. **Second pass:** Reduce the accumulated carry bits (which form much shorter columns, max height 4 even at BW=17).

Counterintuitively, COMBA-CS produces ∼2× more clauses than Dadda, yet is up to 30× faster. The extra auxiliary variables from popcount create intermediate "summary" variables representing the count of 1-bits in groups of 2, 4, 8. These act as abstractions the solver reasons about at a higher level. Formula size is not the right optimization target—propagation structure is.

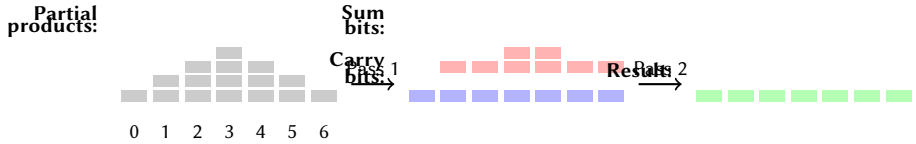

Figure 1: The COMBA-CS two-pass encoding for a 4-bit multiplier. Pass 1 reduces each column independently via popcount (no inter-column carries). Pass 2 reduces the short carry columns (max height 2 here, max 4 at BW=17).

5.1. Adaptive Encoding Selection

While popcount benefits commutativity checking (two identical-structure multiplications), it hurts problems with 3+ independent multiplications where BVE needs carry chains for cascading elimination. We measured that shift-add achieves 106% BVE elimination on associativity problems—through resolution, BVE creates new variables that are themselves eliminated in subsequent rounds, so the cumulative count exceeds the original variable count—while carry-save encodings achieve only 58%.

The adaptive heuristic makes a two-way decision per multiplication. It selects *popcount* when the formula has ≤ 2 symbolic multiplications of the same width—the commutativity pattern where congruence closure discovers equivalent gates between identical multiplier circuits. It selects *shift-add* otherwise: when the formula has 3+ multiplications, or when widths differ. A pre-scan counts symbolic multiplications before encoding begins; if the count exceeds 2, popcount is disabled globally. For constant multiplication with few partial products, COMBA-CS falls back to Dadda-style carry-save reduction. The multiplication count threshold is insensitive: changing it from 2 to 1, 4, or 99 produces identical results because the Gröbner basis solver (the algebraic solver) subsumes the threshold's purpose for polynomial benchmarks, and non-polynomial benchmarks in our suite have ≤ 2 multiplications.

DRAT proof analysis confirms the benefit of column independence:

Table 2
DRAT proof comparison, commutativity BW=11 (CADICAL).

Metric	Std. Comba	COMBA-CS	Change
Proof steps	125,956	57,295	-55%
Avg clause size	11.5	9.3	-19%
Input var involvement	30%	22%	-27%
Fixed variables	124	216	$+74\%$

COMBA-CS produces 55% smaller proofs with 74% more unit propagation during search. The increase in fixed variables means COMBA-CS enables the solver to determine more variable values through unit propagation alone, reducing the search space. This confirms that column independence reduces inter-column reasoning in the proof.

6. Alternative Encodings

Applying the proof-guided methodology to other encoding strategies, we implemented and evaluated three alternatives.

6.1. Booth Radix-4

Booth encoding [13] examines multiplier bits in pairs, generating partial products with values in $\{-2, -1, 0, +1, +2\}\times$ multiplicand. This halves the number of partial products at the cost of two's complement correction.

For constant multiplication (e.g., $x \times 15$), many Booth digits are zero, eliminating entire partial products. On strength_chain_16 ($x \times 15$, $x \times 17$, $x \times 255$):

Table 3
Booth radix-4 vs. shift-add on constant multiplication (CaDiCaL).

Encoding	Vars	Clauses	Conflicts	Decisions	Props	Time
Booth	**496**	**1,780**	**4,071**	**5,144**	**85K**	**0.06s**
block4	983	3,842	18,590	29,890	840K	0.4s
Dadda	756	3,036	29,188	47,431	1.6M	0.7s
shift-add	714	2,910	39,757	49,128	2.1M	1.0s
comba-cs	714	2,906	43,804	54,232	2.4M	1.4s
sortnet	13,316	40,462	34,985	64,579	25M	6.5s

Booth produces 30% fewer variables and 10× fewer conflicts. Proof analysis confirms: Booth's proof is 277 KB vs. ∼10 MB for shift-add—the solver requires fundamentally less reasoning.

6.2. 4-Bit Block Multiplication

We divide both operands into 4-bit sections and compute $4 \times 4 \to 8$-bit sub-products using shift-add (small, propagation-friendly), then accumulate block products via Dadda tree. This creates intermediate variables at 4-bit boundaries that BVE can eliminate efficiently: block4 achieves the highest BVE elimination rate (770 variables, 78% of its total).

On mul_ineq_12 (inequality constraints), block4 is the fastest encoding (13,617 conflicts vs. 15,662 for shift-add), suggesting that the balanced tree structure from 4-bit blocks aids BVE on problems with mixed arithmetic and comparison operations.

6.3. Sorting Network (Negative Result)

We compute n^2 partial product bits, sort each column using a bitwise sorting network (compare-and-swap = $(x \vee y, x \wedge y)$), and add auxiliary constraints enforcing the sorted property ($out[i] \Rightarrow out[i-1]$). The sorted representation makes carry structure explicit.

Despite achieving the highest fixed-variable count (2,915 vs. 217 for shift-add)—confirming excellent propagation from the auxiliary constraints—the encoding produces 13,316 variables and 40,462 clauses (18× and 14× more than shift-add). The 25M propagations overwhelm the structural benefit. This is a clear negative result: auxiliary constraints that improve propagation locally can be net-negative when they dramatically increase formula size.

6.4. No Single Encoding Dominates

Table 4
Best encoding per benchmark class.

Problem class	Best encoding	Key metric
Commutativity (2 identical circuits)	comba-cs	congruence closure
Constant multiplication	Booth	fewer partial products
Circuit equivalence (vs. shift-add)	shift-add	structural matching
Inequality constraints	block4	BVE elimination

The optimal encoding depends on problem structure, not just multiplication count. This motivates encoding selection based on deeper structural analysis of the verification problem.

7. Evaluation

We ran all experiments on an Intel Xeon Platinum 8124M (3.0 GHz, 36 cores) with 68 GB RAM, Ubuntu 24.04, GCC 13.3.0. Solver versions: CaDiCaL 3.0.0, MiniSat 2.2.1, MergeSat (docker-solvers branch, MiniSat-compatible API), CryptoMiniSat 5.11.21. We report median of 3 runs with a 120 s timeout and 6 GB memory limit. Unless otherwise noted, we use the g-only adder encoding (the new default; see Appendix A).

identities (commutativity, overflow, strength reduction: 15 benchmarks division-by-constant: 6), matrix/MAC (trace, dot product commutativity: 4), division/FP (roundtrip, FP add/mul: 6), factoring

7.1. Multiplier Encoding Comparison

Table 5
Four multiplier encodings across four solvers, g-only adder (seconds, T/O = 120 s). Best per row in bold.

Solver	Benchmark	shift	Dadda	Std. Comba	comba-cs	Speedup
CaDiCaL	comm 9	2.29	0.59	0.32	**0.13**	18×
CaDiCaL	comm 11	68.6	3.59	1.80	**0.96**	71×
CaDiCaL	comm 13	T/O	T/O	8.79	**3.39**	∞
CaDiCaL	mat. trace	34.4	3.16	**1.34**	1.93	18×
MiniSat	comm 9	**6.62**	8.99	11.7	6.67	—
MiniSat	comm 11	T/O	T/O	T/O	T/O	—
MiniSat	mat. trace	T/O	61.1	**5.39**	17.3	∞
MergeSat	comm 9	7.44	7.23	3.76	**2.85**	2.6×
MergeSat	comm 11	T/O	22.7	31.3	**14.0**	∞
MergeSat	comm 13	T/O	T/O	65.9	**16.2**	∞
MergeSat	mat. trace	44.1	14.6	**5.35**	8.70	5.1×
CryptoMiniSat	comm 9	10.2	**7.90**	10.1	9.19	1.1×
CryptoMiniSat	comm 11	T/O	102	66.7	**62.0**	∞
CryptoMiniSat	mat. trace	T/O	35.0	**32.5**	88.7	∞

The "Speedup" column shows comba-cs vs. shift-add (the previous default). A clear encoding hierarchy emerges for commutativity: shift-add < Dadda < standard Comba < comba-cs, consistent across all four solvers. For matrix trace, standard Comba is slightly faster than comba-cs on CaDiCaL (1.34 s vs. 1.93 s) because the the carry-save second pass in comba-cs creates slightly different variable structure than standard Comba's immediate carry propagation.

7.2. Cross-Solver Analysis

The four solvers differ in their response to comba-cs for identifiable reasons. CaDiCaL benefits most (71×) because it combines the structural improvement with inprocessing BVE that exploits comba-cs's popcount intermediate variables, plus congruence closure that identifies equivalent gates between the two multiplier circuits. MergeSat benefits from the structural improvement alone (2.6–∞×): as a MiniSat fork with CHB activity and phase saving but no inprocessing, the carry-save structure is sufficient to solve comm BW=11 and BW=13 (both T/O with shift-add). MiniSat's SatELite preprocessor achieves greater simplification on the compact shift-add formula for small problems (comm BW=9: 6.62 s vs. 6.67 s), but on larger formulas the structural benefit dominates (matrix trace: 5.39 s where shift-add times out). Notably, MiniSat times out on comm BW=11 for *all* encodings—the problem exceeds its capacity regardless. CryptoMiniSat is consistently the slowest on multiplication; its native XOR handling provides no benefit (confirmed by running with and without XOR constraints), and it lacks both inprocessing BVE and congruence closure.

7.3. Scaling

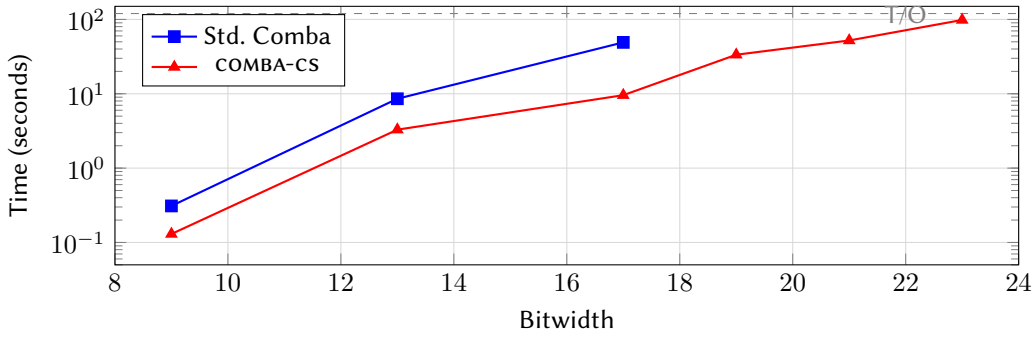

Figure 2: Commutativity scaling (CaDiCaL, ripple adder). Standard Comba times out at BW≥19; COMBA-CS extends the solvable frontier to BW=23. The speedup accelerates with bitwidth.

7.4. Industrial Benchmarks

Table 6
Industrial benchmarks: shift-add vs. COMBA-CS, g-only adder (CaDiCaL).

Benchmark	shift-add	COMBA-CS	Speedup
MurmurHash3 determinism	10.0s	7.2s	1.4×
$\mathrm{tr}(AB) = \mathrm{tr}(BA)$	34.4s	1.9s	18×
MAC dot product comm	11.8s	4.3s	2.8×
keyed hash determinism	1.3s	6.5s	0.2×
div-by-const optimization	9.7s	9.7s	1.0×

COMBA-CS provides large speedups on benchmarks with multiple symbolic×symbolic multiplications (matrix trace, MAC). The keyed hash regression (1.3 s → 6.5 s) is from constant multiplication where the adaptive fallback routes through dadda-cs instead of shift-add; the benchmark remains fast in absolute terms.

More broadly, UNSAT problems show consistent encoding sensitivity (up to 71×) with stable ranking across solvers. SAT problems (factoring) show encoding sensitivity at large bitwidths but with unpredictable ranking—different encodings make different solutions easy to find.

7.5. SMT-LIB QF_BV Benchmarks

To assess external validity, we evaluate on 44 QF_BV benchmarks in SMT-LIB format using CBMC's `smt2_solver` with CaDiCaL. The `smt2_solver` lacks CBMC's word-level simplification,

The "default" column uses the adaptive COMBA-CS heuristic, which selects popcount or shift-add per multiplication. increased from 606/1728 (baseline) to 1320/1728 (+714 benchmarks), where 1728 = 36 benchmarks × 4 multiplier encodings × the total benchmark count is 44. The adaptive heuristic (Section 5) and pre-scan eliminate all regressions except hw_equiv_12, where a single `bvmul` is compared against a manually-constructed shift-add circuit—no encoding heuristic can detect this without knowing the comparison target's structure.

8. Negative Results

A significant portion of this investigation produced negative results. We report them to save future effort and because the failure modes illuminate SAT solver behavior. A natural first question is whether asymptotically superior algorithms—Karatsuba [14] ($O(n^{1.585})$), Toom-Cook [15] ($O(n^{1.465})$),

Table 7
QF_BV benchmarks with adaptive COMBA-CS (CADICAL, seconds).

Category	Benchmark	shift	default	Speedup
COMBA-CS wins (2-mul commutativity, popcount)				
	comm_8	0.69	**0.09**	8×
	comm_16	T/O	**5.3**	∞
	comm_20	T/O	**33.7**	∞
	bf16_mul_v2	22.9	**11.0**	2.1×
Matched (adaptive selects shift-add)				
	overflow_16	0.34	0.34	1.0×
	checked_mul_16	0.39	0.39	1.0×
	assoc_8	27.2	26.7	1.0×
	distrib_8	83.3	81.6	1.0×
Remaining regression (inherent)				
	hw_equiv_12	**14.9**	101	0.15×

or Schönhage-Strassen [16] ($O(n \log n \log \log n)$)—produce better SAT encodings. CBMC includes implementations of all three. They are designed for *computational* efficiency (fewer arithmetic operations), not *verification* efficiency (fewer SAT conflicts). Karatsuba's combination step requires additions with carry chains—exactly the structure Section 3 identifies as the hardness source.

Table 8
Subquadratic encodings vs. std. Comba (CADICAL, seconds).

Benchmark	Std. Comba	Karatsuba	Toom-Cook	Schönhage
comm BW=9	**0.26**	5.54	2.48	86.4
comm BW=13	**8.08**	10.0	31.4	>300
prime BW=28	**3.09**	69.7	71.7	>120
aws_mul	39.4	**3.89**	>120	>120

Karatsuba is competitive on commutativity at BW=13 but 22× slower on primality. Schönhage-Strassen produces 43K variables for BW=7 (vs. ∼600 for Comba)—designed for thousands of bits, it is orders of magnitude too expensive at practical bitwidths. Note that standard Comba loses to Dadda on aws_mul (39.4 s vs. 1.29 s), where constant multiplication dominates; COMBA-CS's adaptive fallback handles this.

8.1. Approaches That Hurt Performance

Several approaches that add structure to the encoding actually hurt performance because they reintroduce carry chains. Radix-4/8 multipliers are 2–15× slower: pre-computing $3x$, $5x$, $7x$ adds carry chains that dominate at practical bitwidths. Full-adder tree popcount is 3× slower than pop0 despite fewer variables, because it creates longer carry chains ($\log_2 n$ bits vs. 2–4 bits). Non-propagation-complete carry merge causes a 25× regression: encoding carry with 4 clauses instead of 6 violates propagation completeness [6].

Sharing structure between multiplications also hurts: caching AND gates between multiplications is 89% slower because shared partial product variables prevent independent BVE—the solver cannot eliminate a variable that appears in clauses from both multipliers.

8.2. Approaches With Limited or Chaotic Benefit

XOR-based techniques consistently disappoint. We tested offline Gaussian elimination, online incremental GE, and CRYPTOMINISAT's built-in XOR handling. The best result was 1.4× on one benchmark, with

neutral or negative results on all others. The problem is that CBMC's XOR constraints are 3-variable gates—too short for GE to combine usefully.

Solver tuning and abstraction refinement fail because the problem is structural, not parametric. We tested 512 CaDiCaL option combinations (16 options $\times$ 4 encodings $\times$ 8 benchmarks); no option changed any result by more than $\pm 3\%$. CBMC's existing CEGAR loop for multiplication (`--refine-arithmetic`) starts with $x \cdot 0 = 0$ and $x \cdot 1 = x$, then adds the full multiplier on the first spurious counterexample. On UNSAT benchmarks, the full multiplier is always needed, so the refinement loop adds 2+ solver calls of overhead (commutativity BW=11: 4.0 s vs. 0.9 s direct comba-cs). All five parallel prefix adders (BK, Kogge-Stone, Sklansky, Han-Carlson, Ladner-Fischer) produce identical learned clause quality (95% glue-1 [17]); BK wins solely by having the fewest variables (Appendix A).

9. Related Work

Algebraic and polynomial methods. Bryant [5] showed that BDD-based multiplication verification requires exponential size. Clegg et al. [18] proved that the Gröbner proof system can be exponentially more powerful than resolution, providing theoretical justification for why our algebraic solver outperforms resolution-based SAT solving on polynomial equation benchmarks. Biere, Kauers, and Ritirc [19] showed that gate polynomials and field polynomials of an arithmetic circuit form a Gröbner basis when variables follow reverse topological order, enabling column-wise verification of multipliers. Kaufmann [20] extended this with SAT-based variable elimination to simplify the Gröbner basis before reduction, and showed that SAT proofs (DRUP) and algebraic proofs (PAC) are interconvertible—connecting the two proof systems that our layered approach uses separately. Kaufmann and Biere [21] built on this in AMulet2, using XOR slicing to separate XOR-linear parts (full-adder sums, handled by Gaussian elimination over GF(2)) from nonlinear parts (carry generation, handled by Gröbner bases). A key difference is the coefficient domain: their approach works over $\mathbb{Q}$ with field polynomials $x^2 - x = 0$ to enforce boolean values, while ours works directly over $\mathbb{Z}_{2^d}$ where modular arithmetic is native—avoiding the exponential blowup from field polynomials at the cost of incompleteness for non-unit constants. Yu et al. [22] achieve linear-time verification of gate-level multipliers via function extraction (backward rewriting through gate polynomials), scaling to 512-bit designs. monomial problem (intermediate polynomial explosion from carry propagation) via reverse engineering of atomic arithmetic blocks. Our approach operates at the word level ($a \times b$ as a single polynomial), making it domain-agnostic but unable to exploit circuit-specific structure. over $\mathbb{Z}_{2^d}$ to solve the equational theory of bit-vectors algebraically. We build on their algebraic framework and extend it with BMC-specific techniques (the algebraic solver).

$n = \mathrm{SF}(2^m)$, the least k with $2^m | k!$). Their approach handles the case where $f - g \neq 0$ as a polynomial but vanishes as a function—e.g., $4x^2 + 4x \equiv 0 \pmod 8$—which our Gröbner basis cannot detect (it only finds ideal membership in the polynomial ring). We confirmed this experimentally: on the image rejection benchmark from [23] ($f : \mathbb{Z}_{2^{12}} \times \mathbb{Z}_{2^8} \to \mathbb{Z}_{2^{16}}$), our Gröbner basis times out while Bitwuzla solves it in 7 ms via word-level brute-force evaluation. Their technique is complete for polynomial equivalence but does not handle non-polynomial operations (bitwise, shifts, inequalities). The two approaches are complementary: our Gröbner basis handles arbitrary polynomial systems with the completeness for the equivalence fragment.

Encoding and SAT techniques. Dadda [8] and Wallace [24] proposed tree-based partial product reduction. Comba [9] proposed column-wise reduction. comba-cs extends Comba with carry-save separation, optimized for SAT solving rather than hardware implementation. Howe et al. [25] decompose univariate binary polynomials into subpolynomials by fixing input bits one at a time, exploiting the fact that coefficients are multiplied by powers of 2 at each level and thus vanish rapidly. Combined with degree reduction compact bit-blasting encodings for polynomial evaluation. The technique is developed for univariate polynomials; extending it to multivariate polynomials (such as $a \times b$) yields the standard partial product structure, since fixing one bit of one operand produces a subpolynomial that still contains

a full multiplication of the remaining bits. Eén and Biere [4] introduced SatELite preprocessing with BVE. Biere et al. [1] extended this to inprocessing in CaDiCaL. Our analysis (Section B) reveals that inprocessing BVE can be counterproductive for specific encoding structures. Brain et al. [6] showed that PC encodings compose for adders but not multipliers. Brain [7] conjectured that a PC multiplier is exponential, and explored Toom-Cook polynomial interpolation as an alternative approximation strategy. Jia et al. [26] improve bit-blasting for nonlinear integer arithmetic (QF_NIA) with Greedy Addition (a Huffman-like ordering that minimizes auxiliary variables) and Multiplication Adaptation (starting with narrow bit-widths). We tested Greedy Addition across four solvers and found mixed results: CaDiCaL benefits (1.4–2.6× faster) because its inprocessing BVE works better with balanced trees, but MiniSat and MergeSat regress (up to 1.8× slower) because their preprocessing BVE prefers long sequential carry chains. This solver-dependent effect reinforces our finding that encoding structure interacts deeply with solver-specific techniques. Fazekas et al. [27] extend incremental inprocessing to support Bounded Variable Addition (BVA), the dual of BVE. We found BVA catastrophic for multiplication (all benchmarks T/O when enabled in CaDiCaL). Our g-only technique (Appendix A) is superficially similar but fundamentally different: g-only adds AND gates that *enable* BVE, while BVA adds variables that *prevent* it. Encoding techniques for SAT have been studied extensively [28, 29].

Word-level solvers and complementary approaches. Niemetz et al. [30] implement word-level normalization in Bitwuzla that avoids bit-blasting for algebraic comba-cs: they handle pure polynomial equations without bit-blasting, while comba-cs handles the general case including inequalities, bitwise operations, and overflow checks. Chukharev et al. [31] show that circuit-structure-aware SAT partitioning can solve multiplier equivalence checking instances that are intractable for both approach.

10. Conclusion and Future Work

We presented a proof-guided methodology for SAT encoding design: analyze DRAT proofs to identify structural bottlenecks, then redesign the encoding to eliminate them. This methodology produced comba-cs (2.6–71× speedup on commutativity) and guided our evaluation of Booth radix-4 (17× on constant multiplication), 4-bit blocks, and sorting networks.

Our analysis reveals that carry propagation—not circuit size—is the dominant hardness source, and that no single encoding dominates all problem classes. The optimal encoding depends on problem structure: commutativity benefits from congruence closure (comba-cs), constant multiplication from fewer partial products (Booth), and circuit equivalence from structural matching (shift-add).

Several directions remain open. First, the proof-guided methodology could be automated: an AI system that reads DRAT proofs, identifies bottlenecks, and proposes encoding modifications would close the loop we perform manually. Second, the encoding selection heuristic could be extended beyond multiplication count to consider deeper structural properties (symmetry, constant operands, comparison targets). Third, propagating encoding flags through the floating-point path would enable Booth and block4 to benefit FP verification.

Declaration on Generative AI. During preparation, the authors used Kiro (an AI coding assistant by Amazon) to assist with implementation, experimentation, and drafting. Kiro is the tool referenced in this declaration, not the second author. All results were verified by the authors, and the final paper was reviewed and edited by the authors.

References

[1] A. Biere, CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT Competition 2020, in: Proc. SAT Competition 2020, 2020, pp. 51–53.

[2] A. Biere, A. Cimatti, E. M. Clarke, Y. Zhu, Symbolic model checking without BDDs, in: TACAS, 1999, pp. 193–207. doi:10.1007/3-540-49059-0_14.

[3] M. Järvisalo, M. J. H. Heule, A. Biere, Inprocessing rules, in: IJCAR, 2012, pp. 355–370. doi:10.1007/978-3-642-31365-3_28.
[4] N. Eén, A. Biere, Effective preprocessing in SAT through variable and clause elimination, in: SAT, 2005, pp. 61–75. doi:10.1007/11499107_5.
[5] R. E. Bryant, On the complexity of VLSI implementations and graph representations of Boolean functions with application to integer multiplication, IEEE Trans. Computers 40 (1991) 205–213. doi:10.1109/12.73590.
[6] M. Brain, L. Hadarean, D. Kroening, R. Martins, Automatic generation of propagation complete SAT encodings, in: VMCAI, 2016, pp. 536–556. doi:10.1007/978-3-662-49122-5_26.
[7] M. Brain, Further steps down the wrong path: Improving the bit-blasting of multiplication, in: SMT Workshop, 2021, pp. 23–31. URL: https://ceur-ws.org/Vol-2908/.
[8] L. Dadda, Some schemes for parallel multipliers, Alta Frequenza 34 (1965) 349–356.
[9] P. G. Comba, Exponentiation cryptosystems on the IBM PC, IBM Systems Journal 29 (1990) 526–538. doi:10.1147/sj.294.0526.
[10] H. S. Warren, Hacker's Delight, 2nd ed., Addison-Wesley, 2012.
[11] A. Haken, The intractability of resolution, Theoretical Computer Science 39 (1985) 297–308. doi:10.1016/0304-3975(85)90144-6.
[12] J. Lv, P. Kalla, Formal verification of Galois field multipliers using computer algebra, in: VLSI Design, 2012. doi:10.1109/VLSID.2012.74.
[13] A. D. Booth, A signed binary multiplication technique, The Quarterly Journal of Mechanics and Applied Mathematics 4 (1951) 236–240. doi:10.1093/qjmam/4.2.236.
[14] A. Karatsuba, Y. Ofman, Multiplication of multidigit numbers on automata, Soviet Physics Doklady 7 (1962) 595–596.
[15] A. L. Toom, The complexity of a scheme of functional elements realizing the multiplication of integers, Soviet Mathematics Doklady 3 (1963) 714–716.
[16] A. Schönhage, V. Strassen, Schnelle Multiplikation großer Zahlen, Computing 7 (1971) 281–292. doi:10.1007/BF02242355.
[17] G. Audemard, L. Simon, Predicting learnt clauses quality in modern SAT solvers, in: IJCAI, 2009, pp. 399–404.
[18] M. Clegg, J. Edmonds, R. Impagliazzo, Using the Groebner basis algorithm to find proofs of unsatisfiability, in: STOC, 1996, pp. 174–183. doi:10.1145/237814.237860.
[19] A. Biere, M. Kauers, D. Ritirc, Challenges in verifying arithmetic circuits using computer algebra, in: SYNASC, 2017, pp. 1–8. doi:10.1109/SYNASC.2017.00009.
[20] D. Kaufmann, Formal Verification of Multiplier Circuits using Computer Algebra, Ph.D. thesis, Johannes Kepler University Linz, 2020.
[21] D. Kaufmann, A. Biere, AMulet 2.0 for verifying multiplier circuits, in: TACAS, 2021, pp. 357–364. doi:10.1007/978-3-030-72013-1_19.
[22] C. Yu, W. Brown, D. Liu, A. Rossi, M. Ciesielski, Formal verification of arithmetic circuits by function extraction, IEEE Trans. CAD 35 (2016) 2131–2142. doi:10.1109/TCAD.2016.2547898.
[23] N. Shekhar, P. Kalla, F. Enescu, Equivalence verification of polynomial datapaths using ideal membership testing, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems 26 (2007) 1320–1330. doi:10.1109/TCAD.2006.888277.
[24] C. S. Wallace, A suggestion for a fast multiplier, IEEE Trans. Electronic Computers EC-13 (1964) 14–17. doi:10.1109/PGEC.1964.263830.
[25] J. M. Howe, M. Brain, A. Gàmez-Montolio, Evaluating binary polynomials using subpolynomials, in: Proceedings of the 23rd International Workshop on Satisfiability Modulo Theories (SMT), volume 4008 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2025, pp. 3–15.
[26] F. Jia, R. Han, P. Huang, M. Liu, F. Ma, J. Zhang, Improving bit-blasting for nonlinear integer constraints, in: ISSTA, 2023, pp. 1230–1241. doi:10.1145/3597926.3598034.
[27] K. Fazekas, F. Pollitt, M. Fleury, A. Biere, Incremental inprocessing rules beyond resolution, in: Pragmatics of SAT, 2025.
[28] G. S. Tseitin, On the complexity of derivation in propositional calculus, in: Automation of

Reasoning, 1983, pp. 466–483. doi:10.1007/978-3-642-81955-1_28.
[29] D. A. Plaisted, S. Greenbaum, A structure-preserving clause form translation, J. Symbolic Computation 2 (1986) 293–304. doi:10.1016/S0747-7171(86)80028-1.
[30] A. Niemetz, M. Preiner, Bitwuzla, in: CAV, 2023, pp. 3–17. doi:10.1007/978-3-031-37703-7_1.
[31] K. Chukharev, I. Gribanova, D. Ivanov, S. Kochemazov, V. Kondratiev, A. Semenov, Effective partitioning method with predictable hardness for CircuitSAT, IEEE Access 13 (2025). doi:10.1109/ACCESS.2025.3527819.
[32] R. P. Brent, H.-T. Kung, A regular layout for parallel adders, IEEE Trans. Computers C-31 (1982) 260–264. doi:10.1109/TC.1982.1675982.
[33] J. Elffers, J. Nordström, Trade-offs between time and memory in a tighter model of CDCL SAT solvers, in: SAT, 2018, pp. 160–176. doi:10.1007/978-3-319-94144-8_10.

A. Adder Encoding Interaction

The top-level adder encoding (used for direct additions and equality checks) interacts with the multiplier encoding. The Brent-Kung (BK) parallel prefix adder [32] produces 95% glue-1 learned clauses on CaDiCaL. All five parallel prefix adders (BK, Kogge-Stone, Sklansky, Han-Carlson, Ladner-Fischer) produce identical glue-1 rates; BK wins because it has the fewest variables (1.9× overhead vs. 2.2–3.0× for others). This gives 16× speedup on addition-heavy UNSAT benchmarks with CaDiCaL.

However, BK *hurts* all other solvers because they lack inprocessing to exploit the glue-1 structure. The extra variables are pure overhead without inprocessing.

Table 9
Adder encoding with comba-cs multiplication (CaDiCaL).

Benchmark	Ripple	BK	g-only
equiv_unsat (pure addition)	0.63s	**0.04s**	0.60s
mul comm BW=9	0.13s	0.13s	0.13s
matrix trace	**0.54s**	1.59s	0.72s

BK is neutral on multiplication (after our adder encoding swap fix that isolates popcount from the top-level adder) but hurts matrix trace. Ripple-carry is the safest solver-independent default.

A.1. g-only BVE Catalyst

Adding redundant AND gates ($g[i] = a[i] \wedge b[i]$) to the ripple-carry adder triggers BVE polarity alignment cascades: the AND gate's clauses structurally match carry generation clauses, enabling BVE to eliminate them trivially, which reduces occurrence counts of input variables, enabling further cascading elimination.

g-only is applied only to *top-level* additions (equality checks, explicit `bvadd`). Multiplier-internal additions are isolated by the adder encoding swap in `unsigned_multiplier` and always use ripple-carry.

Table 10
g-only vs. ripple-carry (shift-add multiplier).

Benchmark	Solver	ripple	g-only
strength_chain_16	MergeSat	5.05s	**2.05s**
hw_mul_equiv_12	MiniSat	T/O	**119s**
add_chain_32	CaDiCaL	0.46s	**0.33s**
div_roundtrip_12	MergeSat	11.4s	**10.3s**

g-only is neutral on comba-cs benchmarks (popcount already provides BVE targets) and causes no regressions on any hard benchmark. It is now the default top-level adder encoding in CBMC.

A.2. Division and Floating-Point

Division and floating-point operations show zero encoding sensitivity across all four solvers (FP add commutativity: 4.42–4.43 s for all three adder encodings; division roundtrip: 7.86 s for all multiplier encodings). This is because: (1) division's internal multiplication always uses shift-add; (2) FP's 48-bit mantissa multiplication falls through to shift-add via comba-cs's sparse-constant fallback (>32 bits with few partial products); (3) the FP barrel shifter and rounding logic dominate solving time.

Table 11
Adaptive fallback threshold sensitivity (CaDiCaL).

Configuration	Benchmark	Time	vs. default
PP sparsity threshold (width threshold fixed at 32)			
No fallback (always popcount)	div_by_const	52.3s	5.6× slower
≤width/2	div_by_const	9.08s	1.0×
≤2·width/3 (default)	div_by_const	9.31s	1.0×
No fallback	keyed_hash	0.90s	1.0×
Default	keyed_hash	1.11s	1.0×
Width threshold (PP threshold fixed at 2/3)			
>16	comm BW=11	0.94s	1.0×
>32 (default)	comm BW=11	0.93s	1.0×
>64	comm BW=11	0.93s	1.0×

A.3. Threshold Sensitivity

The PP sparsity threshold is critical: disabling the fallback causes 5.6× regression on div_by_const (wide constant multiplication). The exact threshold value (1/2 vs. 2/3) makes no measurable difference. The width threshold has no effect on our benchmarks because wide sparse constant multiplication is rare; the threshold exists as a safety margin for FP mantissa multiplication (48 bits).

B. BVE Interaction

CaDiCaL's inprocessing BVE is *counterproductive* for comba-cs at BW≥11, adding 4–140% overhead. Disabling BVE (`elim=0`) gives 1.1–2.3× additional speedup. The carry-save structure creates popcount intermediate variables that BVE attempts to eliminate, but the resulting resolvents are harder than the original clauses. We do not disable BVE by default because the adaptive heuristic routes many multiplications to shift-add where BVE is beneficial.

Tracing clause creation and deletion in CaDiCaL reveals that the latest-learned clauses encode carry propagation relationships between adjacent equality check bits. For commutativity ($c = a \times b$, $d = b \times a$, assert $c = d$), the equality check creates XNOR gates $eq[i] = c[i] \Leftrightarrow d[i]$. The late-learned clauses are of the form $(eq[i] \vee eq[i{+}1])$: "if bit i differs, the adjacent bit must agree."

Pre-providing these as encoding clauses achieves 18–52% speedup at BW=10–13. However, the effect is non-monotonic: +59% regression at BW=14 and +94% at BW=16. This is an instance of CDCL sensitivity to clause perturbation [33]: the hints change BVE's elimination order, and the residual problem's difficulty is a non-monotonic function of that order. The hints are enabled in CBMC for 10–13 bit equality checks only (below 10, the effect is negligible; above 13, regressions dominate).